Title

Christopher Erickson

July 2, 2025

Glossary

dim	Dimension of the cell i.e. 1-d, 2d, 3d
spacedim	Dimension of the problem space, i.e. what the cell resides in
celltype	Type of mapping, linear or quadratic.

Table 1: Description of cell properties

Data Format

Constants

dim	Dimension of the cell i.e. 1-d, 2d, 3d

Table 2: Description of cell properties

Functions

0.1 UpdateFlags(const UpdateFlags f1, const UpdateFlags f2)

1 Configuration Header `config.h`

This header defines fundamental types, constants, and helper functions used throughout the `dromon` library.

1.1 Namespace Macros

`#define DROMON_NAMESPACE_OPEN` Expands to `namespace dromon {`.

`#define DROMON_NAMESPACE_CLOSE` Closes the namespace.

1.2 Polynomial Degree Macros

These macros define polynomial orders used in the basis functions:

- `LINEARP`: 0
- `QUADRATICP`: 1
- `CUBICP`: 2

1.3 Enumerations

DoFDirection

```
enum DoFDirection { u_dir, v_dir, w_dir };
```

Represents the direction of the degree of freedom.

CurrentType

```
enum CurrentType { Electric, Magnetic, Nothing };
```

Indicates the type of current.

UpdateFlags

```
enum UpdateFlags {
  update_default   = 0,
  output_active    = 0x000001,
  verbose_output   = 0x000040,
  suppress_comments= 0x000002
};
```

Flags controlling output and verbosity. These can be combined with bitwise operators:

$$f = output\_active \mid verbose\_output$$

RegularityType

```
enum RegularityType {
  SelfSingular,
  FaceSingular,
  EdgeSingular,
  PointSingular,
  FaceNearSingular,
  EdgeNearSingular,
  PointNearSingular,
  Regular
};
```

Classifies singularity types encountered in geometries.

AdjacencyType

```
enum AdjacencyType {
  Disjoint,
  Self,
  Face,
  Edge,
  Vertex
};
```

Specifies how two cells are adjacent.

1.4 Bitwise Operator Overloads

`UpdateFlags` supports bitwise operators:

- `operator|(UpdateFlags, UpdateFlags)`: bitwise OR
- `operator&(UpdateFlags, UpdateFlags)`: bitwise AND

1.5 GeometryInfo Template

```
template <unsigned int dim,
          unsigned int spacedim,
          unsigned int surf_type>
struct GeometryInfo {
  static constexpr unsigned int nodes_per_cell;
  static constexpr unsigned int vertices_per_cell;
  static constexpr unsigned int faces_per_cell;
  static constexpr unsigned int vertices_per_face;
};
```

This struct provides compile-time constants describing geometric properties.

Definition of `nodes_per_cell`:

$$\text{nodes\_per\_cell} = \begin{cases} (surf\_type + 2), & \text{if } dim = 1 \\ (surf\_type + 2)^2, & \text{if } dim = 2 \\ (surf\_type + 2)^3, & \text{if } dim = 3 \end{cases}$$

Other Constants:

$$\text{vertices\_per\_cell} = 2^{dim}$$
$$\text{faces\_per\_cell} = 2 \times dim$$
$$\text{vertices\_per\_face} = 2^{dim}$$

1.6 Stream Operators

The following overloads are provided for formatted output:

```
std::ostream& operator<<(std::ostream&, const DoFDirection&);
std::ostream& operator<<(std::ostream&, const CurrentType&);
```

These allow printing enums as strings.

1.7 Example

```
DoFDirection dir = v_dir;
std::cout << dir; // Prints "v_dir"

UpdateFlags flags = output_active | verbose_output;
```

1.8 Constants Template

```
template <class Real>
struct constants
{
  static constexpr Real PI = 3.1415926535897932...;
  static constexpr Real C0 = 299792458.;
  static constexpr Real EPS0 = 8.8541878128e-12;
  static constexpr Real MU0 = PI * 4.0e-7;
  static constexpr Real ROOT_EPS0MU0_ = 3.335640951073527...e-9;
  static constexpr std::complex<Real> complexj = {Real(0), Real(1)};
};
```

This template defines physical and mathematical constants used in the library.

Members:

`PI` The mathematical constant π to very high precision.

$$\pi = 3.14159265358979323846\ldots$$

`C0` The speed of light in vacuum:

$$c_0 = 299,792,458 \text{ m/s}$$

`EPS0` The vacuum permittivity:

$$\varepsilon_0 = 8.8541878128 \times 10^{-12} \text{ F/m}$$

`MU0` The vacuum permeability:

$$\mu_0 = 4\pi \times 10^{-7} \text{ H/m}$$

`ROOT_EPS0MU0_`

$$\sqrt{\varepsilon_0 \mu_0} \approx 3.335640951073527 \times 10^{-9}$$

This value frequently arises in electromagnetic formulations.

`complexj` The imaginary unit:

$$j = 0 + i$$

stored as `std::complex<Real>{0,1}`.

Usage Example:

```
Real pi = constants<Real>::PI;
std::complex<Real> j = constants<Real>::complexj;
Real eta = sqrt(constants<Real>::MU0 / constants<Real>::EPS0);
```

2 Class Template `DataOut`

The `DataOut` class template handles exporting mesh and degrees-of-freedom data to VTK and auxiliary files.

Template Parameters

`dim` Topological dimension of the cells (1, 2, or 3).

`spacedim` Dimension of the embedding space.

`celltype` Integer code specifying the type/order of the cell.

`CoefficientType` Type for degrees of freedom coefficients (default: `std::complex<double>`).

`Real` Floating point type (default: `double`).

Public Interface

- `DataOut()`
- `DataOut(Mesh*, std::string file_path, UpdateFlags)`
- `DataOut(Mesh*, DoFHandler*, std::string file_path, UpdateFlags)`
- `void attach_mesh(Mesh*)`
- `void attach_dof_handler(DoFHandler*)`
- `void vtk_out()`
- `void add_cell_data(std::vector<double>&, std::string name="default")`
- `void set_flags(UpdateFlags)`

Description

This class provides methods to:

1. Attach a mesh and (optionally) a degrees-of-freedom handler.
2. Store scalar cell data (and potentially node data).
3. Export:
    - VTK files (`.vtk`) for visualization.
    - Plain text files (`.global.dat`) containing metadata.

File Output

VTK File

- Includes:
    - Grid points
    - Cell connectivity
    - Cell types (VTK codes depending on `dim` and `celltype`)
    - Scalar cell data (if added)

Global Data File

- Outputs summary statistics.
- If `UpdateFlags::verbose_output` or `UpdateFlags::output_active` are set, lists DoFs.

Example Usage

```
DataOut<2,2,1> data_out(mesh, "output/myfile", UpdateFlags::verbose_output);
data_out.add_cell_data(cell_scalars, "stress");
data_out.vtk_out();
```

Note on VTK Cell Ordering

Higher-order cells follow VTK conventions. For example, a 2D quadratic cell is ordered as:

```
/**
 *
 * @tparam dim
 * @tparam spacedim
 * @tparam celltype
 *
 * It is important to note how VTK files treat higher order cells. For dim=2, for
example, with a quadratic
 * cell, we have that
 * 3---------6----------2
 * |         |          |
 * |         |          |
 * 7---------8----------5
 * |         |          |
 * |         |          |
 * 0---------4----------1
 * This orientation holds for the biquadratic approximation, as well as the Lagrange
cells.
 */
```

Figure 1: VTK Files

Dependencies

This class depends on:

- `Mesh`
- `DoFHandler`
- `DoFBase`

as well as `config.h`.